Chapter 6 worked solutions

DS06-01 • Search order and representation cost

BFS visits A, B, C, D, E, at distances 0,1,1,2,3. Parents are B←A, C←A, D←B, E←D because B discovers D first. DFS visits A, B, D, C, E, with B←A, D←B, C←D, E←D. After finishing C, it returns to D and explores E. Adjacency lists handle every vertex and edge a constant number of times, giving $\Theta(V+E)$. A matrix scans V columns per vertex, giving $\Theta(V^2)$.

DS06-02 • Two MST algorithms

Sorted selections are BC=1, AB=2, DE=2, CD=3. Four edges complete the tree at total weight 8. If processing continues, BD, AC, CE have endpoints in the same component and would create cycles. Prim chooses AB, BC, CD, DE. The edge C→D=3 improves D's earlier key from B→D=4. For safety of BC, use cut `{B}` versus the other vertices. BC is its lightest crossing edge. Add BC to any MST missing it, then remove a different cycle edge crossing the same cut. That edge has weight at least 1. The exchanged spanning tree is no heavier, proving safety.

DS06-03 • Negative edges versus negative cycles

Before the added edge the graph is acyclic, with distances s=0, b=5, a=1, t=4. Settled Dijkstra first fixes a at 2. It cannot later correct settled a using b's negative outgoing edge, and can report t=5. Bellman–Ford propagates the correct distances within V−1 passes. Adding a→b=1 creates the reachable cycle a→b→a of weight −3. Distances to a, b, t are unbounded below. Without an edge or path returning to s, δ(s, s)=0 remains finite. Only destinations reachable from a reachable negative cycle are affected.

DS06-04 • Floyd–Warshall loop order

For fixed k, split admissible paths into those avoiding k and those using k. The latter split at k into i→k and k→j, with internal vertices from the previous allowed set. Without relevant negative cycles a shortest path can avoid repeated vertices, supporting this decomposition. With i, j outermost, a needed D[i, k] or D[k, j] may not yet represent its required subproblem. For indices 0,1,2,3, take unit edges 0→3,3→2,2→1. One increasing i, j, k in-place pass processes D[0,1] before D[0,2] and D[3,1] are finite and misses distance 3. With k outermost, intermediate 2 first constructs 3→1, then intermediate 3 constructs 0→1.

Chapter 6 worked solutions (continued)

DS06-05 • AOE critical path

E is s=0, a=3, b=2, t=8. Backward computation gives L[t]=8, L[a]=4, L[b]=2 and L[s]=min(4−3,2−2)=0. Activity s→a has slack L[a]−3−E[s]=1, and a→t also has slack 1. The other two slacks are 0. Critical path s→b→t has length 8. Increasing both a-path durations by 1 changes that path from length 7 to 9, so completion becomes 9. The individually stated total slacks share the same available path margin.

DS06-06 • Single-source paths in a DAG with negative edges

Topologically sort the graph. Initialize d[s]=0 and other distances to infinity. In topological order, relax every outgoing edge of each u with finite d[u]. When u is processed, every possible predecessor is already processed. Every source-to-u path ends at one such predecessor, so induction makes d[u] optimal. Negative weights are allowed because an acyclic graph has no negative cycle or arbitrarily repeatable walk. Sorting plus one relaxation per edge costs O(V+E), with O(V) auxiliary space excluding input. A cyclic graph has no complete topological order, so the predecessor-first induction fails.

DS06-07 • Three objectives on one graph and negative initialization

Additive Dijkstra extracts s,q,p,t. After s, labels in (s,p,q,t) order are (0,6,2,∞); after q they are (0,6,2,11); p does not improve 11. Parent(t)=q and s,q,t totals 11. Bottleneck labels start at −∞. After s they are (−∞,6,2,∞); q sets t=9,parent=q; p improves t to min(9,max(6,6))=6,parent=p. Path s,p,t has minimax value 6.

Kruskal selects s–q2,s–p6,p–t6, total 14. Removing a maximum-weight edge 6 from its s–t path gives a separating cut. A lighter crossing edge would replace it and lower MST weight. Every s–t path crosses that cut, so bottleneck is at least 6; the tree path attains 6. Its sum 12 still exceeds the shortest sum 11.

With only edge s–t=−5, the correct bottleneck is −5. Initializing source 0 gives max(0,−5)=0 incorrectly. Empty-path identity −∞ gives −5. Max-extension remains monotone, so minimax label setting remains correct; this does not remove additive Dijkstra’ s nonnegative-weight requirement.

Chapter 6 worked solutions (continued)

DS06-08 • Mandatory edges and alternatives across all optima

At weight 1, D–E is the only quotient edge and is mandatory; components fall from seven to six. At weight 2, the K4 connects A,B,C and the DE component, using three edges; every K4 edge lies on a cycle and is optional. Weight 5 connects the E-containing component with F,G using two edges; all triangle edges are optional. Every MST weighs $1+3\times2+2\times5=17$. Only D–E is mandatory. Threshold counts $n-c\lambda$ prove these counts for every optimum.

Choose T1={DE,AB,BC,CD,EF,FG} and T2={DE,AC,AD,BD,EG,FG}. Their left parts are paths A–B–C–D and C–A–D–B. Each right part spans E,F,G. Joining by DE gives six edges, connected and acyclic, hence two MSTs. Common edges are DE and FG. Their symmetric difference contains all six weight-2 K4 edges and weight-5 EF,EG, weighing $12+10=22>17$. Vertex A has degree 3 in the difference, excluding a decomposition into vertex-disjoint paths/cycles, whose degrees are at most 2. Unlike matchings, arbitrary spanning trees impose no degree-2 restriction on their symmetric difference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 6 worked solutions (continued)

Verification code appendix: shortest_paths.cpp

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <random>
#include <stdexcept>
#include <utility>
#include <vector>

using Distance = long long;
constexpr Distance infinity = std::numeric_limits<Distance>::max() / 4;

struct Edge {
        int from;
        int to;
        int weight;
};

struct Graph {
        int vertices;
        std::vector<Edge> edges;
};

void validate(const Graph& graph) {
        if (graph.vertices <= 0 || graph.vertices > 1000 || graph.edges.size() > 1000000) {
                throw std::invalid_argument("example supports 1..1000 vertices and at most
1000000 edges");
        }
        for (const auto& edge : graph.edges) {
                if (edge.from < 0 || edge.from >= graph.vertices || edge.to < 0 ||
                        edge.to >= graph.vertices || edge.weight < -1000000 || edge.weight >
1000000) {
                        throw std::invalid_argument("invalid endpoint or weight");
                }
        }
}

std::vector<Distance> dijkstra(const Graph& graph, int source) {
        validate(graph);
        if (source < 0 || source >= graph.vertices) {
                throw std::invalid_argument("invalid source");
        }
        std::vector<std::vector<std::pair<int, int>>> adjacent(graph.vertices);
        for (const auto& edge : graph.edges) {
                if (edge.weight < 0) {
                        throw std::invalid_argument("Dijkstra requires nonnegative weights");
                }
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 6 worked solutions (continued)

```
                adjacent[edge.from].push_back({edge.to, edge.weight});
        }
        using Item = std::pair<Distance, int>;
        std::priority_queue<Item, std::vector<Item>, std::greater<Item>> queue;
        std::vector<Distance> distance(graph.vertices, infinity);
        distance[source] = 0;
        queue.push({0, source});
        while (!queue.empty()) {
                const auto [cost, vertex] = queue.top();
                queue.pop();
                if (cost != distance[vertex]) {
                        continue;
                }
                for (const auto& [next, weight] : adjacent[vertex]) {
                        const auto candidate = cost + weight;
                        if (candidate < distance[next]) {
                                distance[next] = candidate;
                                queue.push({candidate, next});
                        }
                }
        }
        return distance;
}

struct BellmanResult {
        std::vector<Distance> distance;
        bool reachable_negative_cycle;
};

BellmanResult bellman_ford(const Graph& graph, int source) {
        validate(graph);
        if (source < 0 || source >= graph.vertices) {
                throw std::invalid_argument("invalid source");
        }
        std::vector<Distance> distance(graph.vertices, infinity);
        distance[source] = 0;
        for (int round = 1; round < graph.vertices; ++round) {
                bool changed = false;
                for (const auto& edge : graph.edges) {
                        if (distance[edge.from] != infinity &&
                                distance[edge.from] + edge.weight < distance[edge.to]) {
                                distance[edge.to] = distance[edge.from] + edge.weight;
                                changed = true;
                        }
                }
                if (!changed) {
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 6 worked solutions (continued)

```
                        break;
                }
        }
        for (const auto& edge : graph.edges) {
                if (distance[edge.from] != infinity &&
                        distance[edge.from] + edge.weight < distance[edge.to]) {
                        return {distance, true};
                }
        }
        return {distance, false};
}

std::vector<std::vector<Distance>> floyd_warshall(const Graph& graph) {
        validate(graph);
        std::vector<std::vector<Distance>> distance(graph.vertices,
                std::vector<Distance>(graph.vertices, infinity));
        for (int vertex = 0; vertex < graph.vertices; ++vertex) {
                distance[vertex][vertex] = 0;
        }
        for (const auto& edge : graph.edges) {
                if (edge.weight < 0) {
                        throw std::invalid_argument("this Floyd oracle supports nonnegative
graphs only");
                }
                distance[edge.from][edge.to] = std::min(distance[edge.from][edge.to],
                        static_cast<Distance>(edge.weight));
        }
        for (int middle = 0; middle < graph.vertices; ++middle) {
                for (int from = 0; from < graph.vertices; ++from) {
                        for (int to = 0; to < graph.vertices; ++to) {
                                if (distance[from][middle] != infinity && distance[middle]
[to] != infinity) {
                                        distance[from][to] = std::min(distance[from][to],
                                                distance[from][middle] + distance[middle]
[to]);
                                }
                        }
                }
        }
        return distance;
}

void require(bool condition, const char* message) {
        if (!condition) {
                throw std::runtime_error(message);
        }
}

int main() {
```

Chapter 6 worked solutions (continued)

```
        const Graph example{4, {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 6}}};
        require(dijkstra(example, 0) == std::vector<Distance>({0, 3, 1, 4}),
                "slide distances differ");
        const Graph negative{4, {{0, 1, 2}, {0, 2, 5}, {2, 1, -4}, {1, 3, 3}}};
        const auto result = bellman_ford(negative, 0);
        require(!result.reachable_negative_cycle &&
                result.distance == std::vector<Distance>({0, 1, 5, 4}), "negative edge result
differs");
        bool rejected = false;
        try {
                dijkstra(negative, 0);
        } catch (const std::invalid_argument&) {
                rejected = true;
        }
        require(rejected, "Dijkstra accepted negative edge");
        Graph cycle = negative;
        cycle.edges.push_back({1, 2, 1});
        require(bellman_ford(cycle, 0).reachable_negative_cycle, "reachable negative cycle
missed");
        const Graph disconnected_cycle{4, {{0, 1, 3}, {2, 3, -2}, {3, 2, 1}}};
        require(!bellman_ford(disconnected_cycle, 0).reachable_negative_cycle,
                "unreachable negative cycle was misreported as reachable");
        std::mt19937 random(20261024);
        int checked = 0;
        for (int trial = 0; trial < 80; ++trial) {
                const int size = 1 + static_cast<int>(random() % 12);
                Graph graph{size, {}};
                for (int from = 0; from < size; ++from) {
                        for (int to = 0; to < size; ++to) {
                                if (from != to && random() % 4 == 0) {
                                        graph.edges.push_back({from, to, static_cast<int>
(random() % 20)});
                                }
                        }
                }
                const auto all = floyd_warshall(graph);
                for (int source = 0; source < size; ++source) {
                        const auto single = dijkstra(graph, source);
                        const auto bellman = bellman_ford(graph, source);
                        require(!bellman.reachable_negative_cycle && single == all[source] &&
                                single == bellman.distance, "shortest path algorithms
disagree");
                        ++checked;
                }
        }
        std::cout << "PASS shortest paths: " << checked
                << " source cases, exact trace, negative edge/cycle boundaries\n";
}
```

Chapter 6 worked solutions (continued)

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study